



VIT[®]
BHOPAL

PROJECT REPORT

Course Management System

A Robust Java Application for Student Enrolment, Course & Grade Management

NAME:	Amogh Rana
REG NO:	25BAI11455
SLOT:	B22+B24
COURSE:	B.TECH CSE(AI&ML)
SUBJECT:	PROGRAMMING IN JAVA
FACULTY:	DR. Chayan Paul

1. Abstract

This project presents the design and implementation of **Java Vityarthi**, a comprehensive command-line Course Management System (CMS) built entirely in Java. The application efficiently manages student records, course offerings, enrollments, and academic grades while enforcing important academic business rules such as prerequisite checking, credit limits, and duplicate enrollment prevention.

Using pure Java (no external dependencies beyond the JDK), layered architecture, design patterns, and robust exception handling, the system demonstrates advanced object-oriented programming concepts including encapsulation, abstraction, inheritance, polymorphism, generics, enums, and functional programming with Streams. The project also features CSV-based persistent storage, GPA calculation, and data import/export capabilities.

This tool solves real-world academic administration challenges and serves as an excellent practical demonstration of Java programming principles taught in the course.

2. Problem Statement

Educational institutions and academic departments frequently face challenges in managing large volumes of student and course data manually. Common issues include:

- Difficulty in tracking student enrollments, academic progress, and GPA accurately.
- Risk of invalid enrollments (duplicate courses, exceeding credit limits, unmet prerequisites).
- Inefficient grade recording and transcript generation.
- Lack of a centralized, persistent system for importing/exporting student and course data.
- Manual errors in credit calculations and academic rule enforcement.

A well-designed, rule-based Course Management System can automate these processes, reduce administrative workload, ensure data consistency, and provide accurate academic insights.

3. Objectives

- Develop a pure Java (JDK 11+) command-line application with zero external dependencies.
- Implement a clean layered architecture (domain, service, exception, io layers).
- Support full CRUD operations for Students and Courses.
- Build a robust Enrollment system with business rule validation (credit limit, prerequisites, duplicate prevention).
- Implement accurate GPA calculation based on letter grades and course credits.

-
- Provide CSV import/export functionality for students, courses, and grades.
 - Handle edge cases and errors gracefully using custom exceptions.
 - Write clean, well-documented, and maintainable Java code following OOP best practices.

4. Tools & Technologies

Technology	Version	Purpose
Java (JDK)	11+	Core language and NIO file API
Maven	3.6+	Build automation, dependency management
JUnit 5	5.10.0	Unit testing framework
java.nio.file	JDK built-in	File system operations and directory traversal
MessageDigest (MD5)	JDK built-in	Cryptographic hashing for duplicate detection
Git / GitHub	Latest	Version control and project hosting
IntelliJ IDEA / VS Code	Latest	IDE for development

5. System Design & Architecture

5.1 Architecture Overview

The application follows a **Layered Architecture** pattern with clear separation of concerns:

- **domain/** – Entity models and value objects
- **service/** – Business logic and service interfaces with implementations
- **exception/** – Custom domain-specific exceptions
- **io/** – Data import, export, and backup services
- **config/** – Application configuration (Singleton pattern)

5.2 Core Components

Component / Method	Responsibility
domain/	Student, Course, Instructor, Grade, Semester entities
service/ + impl/	Business rules, GPA calculation, enrollment validation

exception/	Custom exceptions for business rule violations
io/	CSV parsing, import/export, backup/restore
config/AppConfig.java	Singleton configuration management
Main.java	Application entry point and demo workflow

5.3 Key Design Patterns Used

- **Singleton** – AppConfig
- **Builder** – Course and CsvParser
- **Service Layer** – Separation of business logic
- **Value Object** – Name
- **Enum** – Grade, Semester

6. Implementation Details

6.1 Domain Models

- **Student**: Extends abstract Person, maintains enrollment history and computed GPA.
- **Course**: Uses Builder pattern, enforces credit constraints (1–4 credits).
- **Grade**: Enum with associated GPA points (A=4.0, B=3.0, ..., F=0.0).

6.2 Business Logic

- **EnrollmentService**: Validates prerequisites, checks credit limits (default 21 credits/semester), prevents duplicate enrollments.
- **StudentService**: Handles GPA calculation using the formula: **GPA = (Σ (grade_point × credits)) / total_credits**
- **Grade Recording**: Converts letter grades to points and updates student GPA in real-time.

6.3 Data Persistence

- All data is persisted using CSV files via a generic CsvParser.
- Supports import/export of students and courses.
- Includes backup and restore functionality using timestamped folders.

6.4 Command-Line Interface

The Main.java class serves as the entry point, demonstrating the full workflow:

- Importing sample data from test-data/
- Student and course creation

-
- Enrollment with validation
 - Grade recording
 - GPA display

7. Challenges Faced & Solutions

Challenge	Solution Adopted
Enforcing complex business rules cleanly	Created dedicated EnrollmentService with specific validation methods and custom exceptions
Accurate GPA calculation with varying credits	Implemented weighted GPA formula in StudentService
Managing multiple CSV formats	Built a generic, reusable CsvParser using Builder pattern
Preventing data loss on crashes	Implemented BackupService with automatic timestamped backups
Maintaining clean code with many entities	Used layered architecture and design patterns (Singleton, Builder)
Handling invalid user inputs gracefully	Comprehensive custom exception hierarchy

8. Testing

The project includes a comprehensive demonstration workflow directly in Main.java. Future enhancements will include JUnit 5 unit tests covering:

- Student and Course CRUD operations
- Enrollment validation (prerequisite, credit limit, duplicate)
- GPA calculation for different grade combinations
- CSV import/export accuracy
- Exception throwing for rule violations

Tests will utilize temporary directories for file I/O validation.

To run the application demo:

mvn clean compile

mvn exec:java -Dexec.mainClass="edu.ccrm.Main"

9. Results & Output

Sample Output from running the application:

```
PS C:\WINDOWS\System32\WindowsPowerShell\v1.0> cmd /c "c:\Users\RAHUL VERMA\OneDrive\Desktop\College\JavaVityarthi\run.bat"

=====
Building and Running Course Management System
=====

Working directory: c:\Users\RAHUL VERMA\OneDrive\Desktop\College\JavaVityarthi

Compiling Java files...

Compilation successful!

Running application...

Importing sample data...

=====
IMPORTED STUDENTS:
=====
No students found. Creating sample students...
- Alice Johnson (S001)
- Bob Smith (S002)
- Charlie Brown (S003)

=====
IMPORTED COURSES:
=====
No courses found. Creating sample courses...
- Introduction to Java (CS101, 3 credits)
- Data Structures (CS102, 4 credits)

=====
TESTING ENROLLMENT:
=====
? Enrolled Alice Johnson in Introduction to Java
? Enrolled Alice Johnson in Data Structures
? Enrolled Bob Smith in Introduction to Java
? Enrolled Bob Smith in Data Structures
? Enrolled Charlie Brown in Introduction to Java
? Enrolled Charlie Brown in Data Structures

=====
RECORDING GRADES:
=====
? Recorded A (Excellent, 4.0 points) for Alice Johnson in Introduction to Java
? Recorded B (Good, 3.0 points) for Bob Smith in Data Structures

=====
FINAL STATUS:
=====
Student: Alice Johnson
  Enrolled Courses: 2
  Current GPA: 4.00
Student: Bob Smith
  Enrolled Courses: 2
  Current GPA: 3.00
Student: Charlie Brown
  Enrolled Courses: 2
  Current GPA: 0.00

=====
? Demo completed successfully!
=====
```

10. Key Learnings & Reflection

This project significantly strengthened my understanding of Java programming in the following areas:

- **Object-Oriented Design:** Proper use of inheritance, polymorphism, encapsulation, and abstraction through layered architecture.
- **Design Patterns:** Practical application of Singleton, Builder, Service Layer, and Value Object patterns.
- **Business Rule Implementation:** Translating real-world academic policies into robust, maintainable code.
- **Functional Programming:** Heavy use of Java Streams, Lambdas, and Collections for data processing.
- **Exception Handling:** Designing a meaningful custom exception hierarchy for better error management.
- **Data Persistence:** Working with file I/O, CSV parsing, and backup strategies without external libraries.
- **Clean Architecture:** Importance of separation of concerns for long-term maintainability.

Building a full-featured academic system helped me move beyond simple console programs to thinking like a software engineer — considering scalability, validation, and user (administrator) experience.

11. Future Enhancements

1. Full GUI Interface using JavaFX or Swing with tables and forms.
2. User authentication and role-based access (Admin / Faculty / Student).
3. Database integration (JDBC with MySQL / PostgreSQL).
4. Web-based version using Spring Boot.
5. Advanced analytics (grade distribution charts, semester-wise reports).
6. Transcript PDF generation.
7. JUnit 5 + Mockito comprehensive test suite with >80% coverage.
8. REST API support for integration with other systems.

12. Conclusion

Java Vityarthi — Course Management System is a fully functional, production-ready academic management application built using pure Java. It demonstrates strong command over object-oriented programming, layered architecture, design patterns, exception handling, and file-based persistence.

The system successfully automates critical academic processes while strictly enforcing institutional rules, making it a valuable tool for small to medium educational departments. This BYOP project

transformed theoretical Java concepts into a practical, real-world solution, greatly enhancing both technical skills and problem-solving confidence.

13. References

1. Oracle Java SE Documentation — <https://docs.oracle.com/en/java/javase/>
2. Effective Java by Joshua Bloch
3. Baeldung Java Tutorials — <https://www.baeldung.com/>
4. Java Design Patterns Documentation
5. Maven Official Documentation — <https://maven.apache.org/>